

AI-Powered Multi-Level Sudoku Solver and Evaluator

Complex Computing Problem (CCP)

Course: CSC202 – Artificial Intelligence

Submitted By

Name: Taimoor Ali

Roll Number: 2024_CS_609_(SECTION_A_)

Table of Contents

🧠 AI-Powered Multi-Level Sudoku Solver and Evaluator	1
Complex Computing Problem (CCP)	1
Course: CSC202 – Artificial Intelligence	1
Submitted By.....	1
1. Project Overview	4
2. Course Learning Outcome (CLO) Alignment	4
3. CCP Attributes	4
3.1 Depth of Analysis Required.....	4
3.2 Depth of Knowledge Required.....	4
3.3 Interdependence	5
4. Problem Formulation	5
4.1 Sudoku as a Constraint Satisfaction Problem (CSP)	5
Variables	5
Domains	5
Constraints.....	5
4.2 Difficulty Levels	5
5. System Architecture	6
5.1 Puzzle Generator (puzzle_generator.py).....	6
5.2 Solver Engine (solverlogic.py).....	6
5.3 Backend API (main.py)	6
5.4 Frontend UI	6
6. Algorithm Design and Justification.....	6
6.1 Uninformed Search (Backtracking)	6
Working:	6
Characteristics:.....	7
6.2 Informed Search (MRV + AC-3 Concept)	7
MRV (Minimum Remaining Values).....	7
AC-3 (Arc Consistency)	7
Combined Effect:.....	7
6.3 Constraint Propagation (Forward Checking)	7
6.4 Local Search (Hill Climbing)	8
Cost Function:	8

Behavior:.....	8
Limitation:.....	8
7. Implementation Details	8
7.1 Puzzle Generation.....	8
7.2 Solver Metrics.....	8
7.3 API Design.....	8
7.4 Frontend System	8
8. Performance Analysis.....	9
8.1 Metrics Collected.....	9
8.2 Observations.....	9
8.3 Key Insight.....	9
9. Testing Evidence.....	9
10. Challenges and Solutions	9
10.1 Puzzle Uniqueness	9
10.2 Metric Consistency.....	10
10.3 Frontend-Backend Integration	10
11. Future Improvements.....	10
12. Conclusion	10

1. Project Overview

This project implements an AI-based Sudoku solving and benchmarking system that generates, solves, and evaluates 9×9 Sudoku puzzles across four difficulty levels:

- Easy
- Medium
- Hard
- Expert

The system is built using Python (FastAPI backend) and a web-based frontend interface. It compares multiple Artificial Intelligence search strategies including uninformed search, informed search, constraint propagation, and local search techniques.

The main objective is to analyze and compare how different AI search strategies perform on a Constraint Satisfaction Problem (CSP) under varying levels of difficulty.

2. Course Learning Outcome (CLO) Alignment

- **CLO 3:** Perform uninformed, informed, local, and adversarial search strategies for problem solving.
- **Bloom's Level:** Evaluate (Level 5)

This project satisfies CLO 3 by implementing and evaluating multiple AI search strategies and comparing their performance using measurable metrics.

3. CCP Attributes

3.1 Depth of Analysis Required

Sudoku is modeled as a CSP requiring formal reasoning, heuristic optimization, and algorithmic comparison under different constraints.

3.2 Depth of Knowledge Required

The system applies AI concepts including: - Backtracking search - MRV heuristic - Arc Consistency (AC-3 concept) - Forward checking - Local search (Hill Climbing)

3.3 Interdependence

The system is modular: - Puzzle Generator → Solver → API → Frontend → Benchmarking All components are interdependent; changes in puzzle difficulty affect solver performance and metrics.

4. Problem Formulation

4.1 Sudoku as a Constraint Satisfaction Problem (CSP)

Sudoku is modeled as:

Variables

Each cell in a 9×9 grid (total 81 variables)

Domains

Possible values {1–9} for each empty cell

Constraints

- Each row must contain unique digits (1–9)
- Each column must contain unique digits (1–9)
- Each 3×3 subgrid must contain unique digits (1–9)

The objective is to assign values to all variables such that all constraints are satisfied.

4.2 Difficulty Levels

Level	Clues
Easy	45
Medium	35
Hard	28
Expert	25

Fewer clues increase search complexity and branching factor.

5. System Architecture

5.1 Puzzle Generator (`puzzle_generator.py`)

- Generates a complete valid Sudoku board using backtracking
- Removes cells while maintaining uniqueness
- Produces difficulty-based puzzles

5.2 Solver Engine (`solver`)

Implements four AI strategies: - Backtracking Search - MRV Heuristic Search - Forward Checking (conceptual) - Hill Climbing (Local Search)

5.3 Backend API (`main.py`)

FastAPI endpoints: - `/generate` → generate puzzle - `/solve` → solve puzzle - `/compare` → compare all algorithms

5.4 Frontend UI

- Interactive Sudoku grid
- Algorithm selection
- Difficulty control
- Step-by-step animation
- Benchmark table visualization

6. Algorithm Design and Justification

6.1 Uninformed Search (Backtracking)

Backtracking is a depth-first search algorithm that explores all possible assignments.

Working:

- Select empty cell

- Try values 1–9
- Validate constraints
- Backtrack if invalid

Characteristics:

- Complete but inefficient
- No heuristics
- Baseline algorithm

6.2 Informed Search (MRV + AC-3 Concept)

MRV (Minimum Remaining Values)

Selects the cell with the fewest valid options.

Reduces branching factor by solving hardest variables first.

AC-3 (Arc Consistency)

AC-3 enforces constraint consistency between variables by removing invalid domain values early.

In Sudoku: - Removes impossible values from rows, columns, and boxes - Reduces search space before solving

Combined Effect:

MRV + AC-3 significantly reduces search space and improves performance.

6.3 Constraint Propagation (Forward Checking)

Forward checking removes invalid values from neighboring variables after each assignment.

If a variable has no valid values left → immediate backtracking occurs.

Advantages: - Early failure detection - Reduces unnecessary recursion

6.4 Local Search (Hill Climbing)

Hill Climbing starts with a randomly filled complete board and iteratively reduces constraint violations.

Cost Function:

Measures duplicate values in rows.

Behavior:

- Accepts only improving moves
- Stops when no conflicts remain

Limitation:

- Can get stuck in local minima
- Not guaranteed to find solution

7. Implementation Details

7.1 Puzzle Generation

Steps: 1. Generate full valid Sudoku using backtracking 2. Remove numbers based on difficulty 3. Ensure uniqueness of solution

7.2 Solver Metrics

Each solver tracks: - Execution time - States explored - Backtracks performed - Step-by-step solving animation

7.3 API Design

Endpoints: - POST /generate → returns puzzle - POST /solve → solves puzzle - POST /compare → compares all algorithms

7.4 Frontend System

Features: - Sudoku grid input - Algorithm selection - Real-time solving animation - Benchmark comparison table

8. Performance Analysis

8.1 Metrics Collected

- Execution Time
- States Explored
- Backtracks
- Success Rate

8.2 Observations

- Backtracking → slow due to exhaustive search
- MRV + AC-3 → fastest due to pruning
- Forward Checking → early failure detection improves efficiency
- Hill Climbing → fast but unreliable due to local minima

8.3 Key Insight

Informed search (MRV + AC-3) performs best due to early pruning and reduced branching factor.

9. Testing Evidence

- Tested across all difficulty levels
- Compared all algorithms on same input
- Verified Sudoku constraint correctness
- Logged metrics via backend API and UI

10. Challenges and Solutions

10.1 Puzzle Uniqueness

Ensuring each puzzle has one solution was computationally expensive.

Solution: Backtracking-based validation with controlled removal.

10.2 Metric Consistency

Different algorithms required unified performance tracking.

Solution: Centralized metric system in solver class.

10.3 Frontend-Backend Integration

Handled using FastAPI with CORS support.

11. Future Improvements

- Full AC-3 independent implementation
- Complete forward checking domain tracking
- Add AI adversarial race mode
- Add graph-based performance visualization
- Extend to 16×16 Sudoku (Hexadoku)

12. Conclusion

This project demonstrates an AI-based Sudoku solver and benchmarking system using multiple search strategies. It models Sudoku as a CSP and compares uninformed, informed, constraint propagation, and local search techniques.

Results show that MRV combined with AC-3 significantly outperforms traditional backtracking due to reduced search space and better heuristic guidance.